Smart Campus Parking

Milestone 4: Alpha Release and Software Security

Samarth Thakor and Nirmal Naik

Prepared November 11, 2026 | Due November 12, 2026

GitHub repository:
https://github.com/SamarthThakor/smart-campus-parking
Live AWS application:
https://smartcampusparking.34-233-200-219.sslip.io/

Release status: The alpha is integrated into GitHub main and deployed on AWS Lightsail with trusted HTTPS. Backend and browser CI passed before merge. Nineteen live checks passed. Private administrator account creation remains an operator handoff; live administrator operations have not yet been verified.

1. Alpha Feature Completeness

The alpha extends the Milestone 3 student registration, permit selection, schedule editing and parking recommendations. It adds privacy controls and administrator visibility while strengthening the shared authentication and API layers. Pull request #2 merged the alpha into main at 6d28ea8. The deployed release is e904ced, the tested PR head included in that merge.

New alpha user story	Implemented behavior
As a student, I can retrieve or remove my data.	Download an owner-only JSON export. Delete the live account and classes after password and CSRF confirmation; all associated sessions are revoked.
As an administrator, I can inspect recent changes.	A dashboard history view lists the newest 50 lot updates. Lot availability/status and permit-rule writes each include an audit entry in the same transaction.
As a user, I can sign out reliably.	Server-side token records provide one-hour absolute expiry and immediate logout revocation, including rejection of a replayed signed-out cookie.
As an operator, I can launch a safer alpha.	Production checks require a persistent secret, HTTPS cookies and trusted hosts. Public demo accounts are disabled; a private CLI creates administrators.

Deferred features and technical justification

The requested LIU Post map remains deferred until campus building/lot coordinates and map-asset permissions are verified and the meaning of live updates is agreed. Existing fictional coordinates must not be presented as real campus guidance. Overtime penalties require agreed parking duration, grace-period and charge rules, plus a parking-session model and reliable start/end timestamps. Implementing fines from guessed rules would produce incorrect results. Neither feature is claimed as complete.

2. System Integration and API Design

The same-origin HTML/CSS/JavaScript interface sends JSON to Flask through a shared request helper. The browser supplies the session cookie and CSRF header; Flask applies centralized security checks before domain routes. SQLAlchemy executes parameterized statements against file-backed SQLite. Alembic applies pending versioned migrations before routes serve traffic.

Layer	Responsibility and boundary
Browser / static/app.js	Forms and safe text rendering; refreshes schedule/recommendations after successful writes. No database credentials or password hashes enter the client.
http.py and security.py	JSON parsing/error shape, CSRF, session lookup, role checks, rate limits and response headers.
auth.py, profile.py, schedules.py, parking.py	Account, personal data, class and administrator endpoint responsibilities; owner IDs come from the session.
validators.py and services.py	Class validation and parking rules are independent of HTML layout. Stable API addresses preserve the existing frontend contract.
models.py and migrations/	Relational constraints, transactions and explicit schema revisions; no destructive table recreation during upgrades.

REST contract and data transfer

API group	Methods and response semantics
/api/auth/*	POST register: 201; POST login: 200; POST logout: 204. GET csrf returns a token.
/api/me and /api/me/export	GET current account/export: 200. PATCH permit: 200. DELETE account: 204 after password confirmation.
/api/classes and /api/classes/{id}	GET list: 200; POST create: 201; PATCH edit: 200; DELETE: 204. Only the signed-in owner can access a class.
/api/recommendations	GET returns next class, best eligible lot, up to two alternatives, reason and future-day provisional flag.
/api/admin/*	GET lots/history; PATCH lot state/count; PUT replaces accepted permit IDs. Administrator role required.

Errors consistently use an error object with code and message: 400 invalid input, 401 authentication/expiry, 403 CSRF or role, 404 missing/unowned record, 409 duplicate signup, 413 oversized request and 429 throttling with Retry-After. API responses are not cached; audit history is bounded to 50 records. Lists are not yet paginated beyond that limit. The complete contract is in docs/API_ALPHA.md.

Schema evolution

Revision 0003_alpha_security adds auth_session (token digest, user foreign key, expiry) and auth_limit (keyed identifier, attempt count, window expiry), with expiry indexes. Existing users, classes, permits, lots and audit rows remain intact. Account deletion cascades to classes and sessions. An upgrade test starts from populated revision 0002 and verifies preserved data. Existing browser sessions must sign in again after this upgrade.

3. Security and Privacy Implementation

Authentication and authorization

Passwords use Werkzeug hashing. Signup accepts only email and password and cannot assign an administrator role. A private create-admin command uses a hidden, confirmed password prompt and refuses to promote an existing account. Login rotates a random session token; only its SHA-256 digest is stored in the database. Sessions expire after one hour, and logout deletes the current server record. Class operations and exports use the authenticated owner rather than trusting a supplied user ID.

Abuse resistance and input defenses

Login, registration and password-confirmed deletion use persistent 15-minute fixed-window limits: eight attempts per account and forty per source address. Counts include successful attempts and survive restart. Atomic updates are tested concurrently. The counter key is an HMAC of the identity/window, so the table does not retain raw attempted emails or IP addresses. Unknown accounts perform a dummy password-hash verification; login errors are generic.

All modifying API requests require a CSRF token. Validation checks JSON shape, types, lengths, weekday ranges, time ordering, referenced records and allowed fields. Requests are limited to 16 KiB. SQLAlchemy parameterizes values. The interface inserts user text using textContent; the browser test confirms that an HTML-like class title stays text and creates no image element or executable event handler.

Browser protection and transmission

The content-security policy permits same-origin resources, blocks inline script and disallows framing, objects and external form targets. Responses include nosniff, no-referrer and restricted geolocation/camera/microphone permissions. API responses use no-store. Cookies are HttpOnly and SameSite=Lax. Production additionally requires Secure cookies and trusted hostnames and sends HSTS. AWS nginx terminates HTTPS with a trusted Lets Encrypt certificate. Live checks confirmed HSTS, CSP, no-store and Secure/HttpOnly cookies; a certificate-renewal dry run passed.

Privacy and data lifecycle

The alpha stores email, password hashes, permit choice, weekly classes and administrator audit records. It does not collect GPS locations, license plates, payment details or university credentials. Students can export their own records or delete their live account, classes and sessions after password confirmation. A public privacy notice explains retention and hosting limits. Database-error logs omit SQL parameters and credentials. Expired session/counter rows are cleaned on authentication activity; administrator history remains retained for the alpha.

Residual risks and limits

SQLite is not application-encrypted; a deployed volume and backups need restricted access and verified provider controls. Deletion does not instantly remove old backups. Signup still reveals duplicate emails through HTTP 409. Email verification, password recovery and MFA are not implemented. Forwarding headers are intentionally untrusted, so users behind one proxy may share its address limit. A stolen live cookie remains usable until expiry or revocation. These controls reduce specific risks; they do not constitute a penetration-test certification.

4. Technical Debt and Refactoring Reflection

Milestone 3 concentrated authentication, security checks, class validation, student routes and administrator operations in app/routes.py. The alpha removes that concentration while retaining its endpoint contract and the parking algorithm. This is a structural change supported by regression tests, not a claim that all technical debt is eliminated.

Before	Alpha change	Maintenance benefit
One route module mixed unrelated responsibilities.	Focused auth, profile, schedule and parking modules; routes.py only registers them.	A domain change has a clear location and fewer unrelated concerns.
Class validation lived inside HTTP routing.	validators.py owns class-field and reference checks.	The API and domain validation can evolve independently of page markup.
A valid signed cookie had no server revocation record.	Persistent token digests and expiry in auth_session.	Logout and account deletion have directly testable revocation behavior.
Audit records were not visible in the dashboard.	Bounded administrator history endpoint and view; permit-rule writes also audited.	Operators can inspect saved changes without database access.
Compressed statements obscured control flow.	Conventional Python formatting across application modules and tests.	Transactions, validation and error paths are easier to review.
Demo setup was unsuitable for shared hosting.	Production guards, account-free catalog initialization and private admin creation.	Deployment need not expose publicly documented demo credentials.

Verification of the refactoring

Check	Recorded local result
Backend tests	60 passed; zero failures and errors. Includes the original 42 checks and 18 alpha checks.
Statement coverage	94.68% (569 of 601 statements); 85% minimum enforced.
Real-browser workflow	21 checks passed in Chrome, including restart persistence, lot closure, owner export, deletion, safe HTML-like text and mobile layout.
Dependency audit	26 installed runtime/development packages scanned; no known vulnerabilities reported at scan time.
Execution scope	Local Windows/SQLite tests plus passing GitHub Ubuntu 24.04 backend/browser CI. Nineteen additional live HTTPS checks passed on AWS.

Evidence files: docs/ms4-evidence/test-summary.json, browser-result.json and dependency-audit.json. GitHub Actions run 43102887970 passed both backend and browser checks before merge. Cloud results are recorded in cloud-result.json. Historical Milestone 3 results remain separate.

Remaining debt

The frontend still uses one JavaScript file; broader feature growth should split it into modules. SQLite supports the single-instance alpha, not multiple independent web instances. A shared database, coordinated migrations, trusted proxy configuration, backup retention, richer account recovery and verified campus data remain future work.

Cloud Release and Verification

Public alpha: smartcampusparking.34-233-200-219.sslip.io
Deployed November 11, 2026. Locations and availability remain fictional; use test data only.

Assignment requirement	Verified state
Alpha features integrated into main	PR #2 merged successfully at 6d28ea88b7fd1c8835babcba1ab55164bffb5a1d after CI passed.
System integration and API design	Stable JSON API, modular responsibilities and populated-database migration checks documented.
Security and privacy implementation	Local regression evidence plus trusted hosted HTTPS, secure cookies, access controls, export and deletion checks.
Technical-debt reflection	Concrete before/after changes and regression evidence documented in section 4.
Updated cloud alpha	AWS Lightsail in us-east-1a; deployed release e904ced5c55e1b8858c85efa8363659128d44cc9. Public login page verified.

Hosting and cost boundaries

One Ubuntu 24.04 instance provides 1 GB RAM and 40 GB persistent SSD. Gunicorn listens only on loopback behind nginx. SQLite and the persistent secret remain on the host; SSH access is limited to the Lightsail browser client. A static IP and free sslip.io name avoid domain registration. Automatic certificate renewal checks every six hours; the renewal dry run succeeded.

The account was verified as AWS Free plan with $100 credits and 182 days remaining. The selected server consumes credits at a listed $7/month. The user authorizes free credits only: no paid-plan upgrade or card charges. Free access ends when credits or the plan period expire; this is not permanent free hosting. Export necessary data before that deadline.

Observed live checks

Nineteen checks passed: health, trusted HTTPS, response headers, Secure/HttpOnly cookies, student registration/login, catalog loading, permit update, class creation, recommendations, administrator-role denial, CSRF rejection, export and password-confirmed deletion. Class data and the authenticated session survived a service restart. The disposable test account was deleted after verification. nginx and the application service both reported active.

Administrator handoff and remaining limitations

No public demo administrator is seeded. The private create-admin command is prepared in the AWS terminal, but the operator must enter and confirm their own password. The last database check found zero administrator accounts. Live administrator lot changes/history therefore remain unverified; those workflows passed local and hosted CI tests. After creating the administrator, sign in and confirm a lot update appears in history. No password is included in this report or source package.

Sources and evidence

MS4.pdf, pages 1-2; CS690_Milestone3_Team3.pdf; Smart_Campus_Parking_Read_Along.pptx. Test summaries: docs/ms4-evidence/. Live student checks: cloud-result.json. Source release: e904ced; main merge: 6d28ea8.

Merged PR #2 | Passing CI | AWS Free plan limits | Free hostname